

Answer key — finished scripts and challenge solutions.

Welcome to Scratch D - the big one. The last three sheets each built a small finished project. This time the four projects are four stages of *one* game: **Maze Quest**. You steer a player through a maze without touching the walls, dodge a roaming enemy, and race to the goal. The new idea that ties many sprites together is the gold **broadcast** message and its partner **when I receive**: one sprite shouts a message, and every sprite (and the stage) that is listening reacts. That is how a game wins, loses, and changes screens. Open scratch.mit.edu, click *Create*, and let us build it.

The block groups you will use. Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

Motion	(blue)	Move the sprite: move, turn, go to, point towards, change x or y.
Looks	(purple)	Change how the sprite looks: say, costume, size, backdrop.
Sound	(pink)	Play a sound.
Events	(yellow)	Start a script (green flag, key, click, when I receive) and broadcast messages.
Control	(orange)	Choose when and how often blocks run: wait, forever, repeat, if, stop.
Sensing	(light blue)	Ask about the world: touching?, key pressed?, mouse, ask and answer.
Operators	(green)	Do maths and comparisons: plus, pick random, less-than, equals, more-than, and/or.
Variables	(dark orange)	Remember a value like score: set, change, and show a variable.

Project 1: The maze and the walls

Draw a maze and steer the player through it. The walls are electric: if the player touches the wall colour, it jumps back to the start. One forever loop reads the arrow keys and watches for walls.

Get ready. Click the *Stage*, then *Backdrops*, and paint a maze: thick **black** walls with a clear path from the bottom-left *start* to the top-right. Rename the cat sprite to **Player** (click its name and retype). The player is small, so it fits the corridors.

New blocks.

`touching color [black]?`

True when the player touches the wall colour. Click the patch and pick your wall colour.

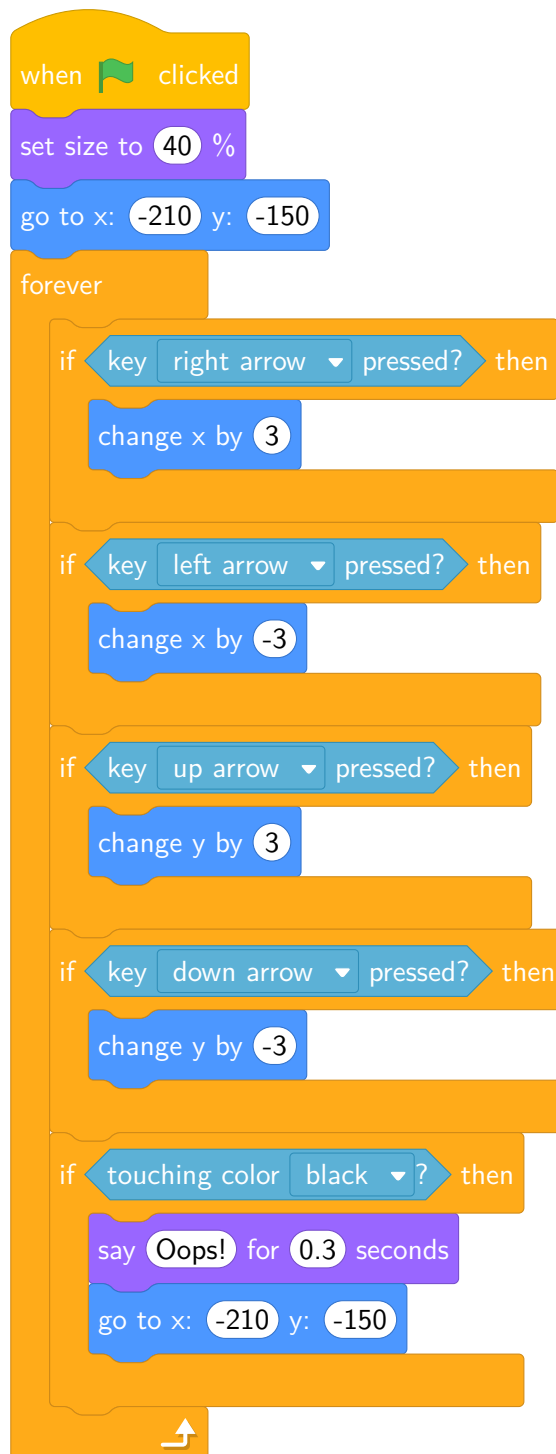
`if <key [] pressed?> then change x/y`

Reading the four arrow keys inside one forever loop steers the sprite smoothly.

Build it.

1. On the Player, start with `when green flag clicked`, then `set size to (40) %` and `go to x: -210 y: -150` (your start corner).
2. Add a `forever` loop.
3. Inside it, add four choices, one per arrow key: `if <key [right arrow] pressed?>` then `change x by (3)`; the same for **left arrow** with `change x by (-3)`, **up arrow** with `change y by (3)`, and **down arrow** with `change y by (-3)`.
4. Below those, still in the loop, add `if <touching color [black]?>` then `say (Oops!) for (0.3) seconds` and `go to x: -210 y: -150`.
5. Green flag, and steer with the arrow keys. Brush a wall and you snap back to the start.

When you are done, your code should look like this:



Reading keys with `if <key () pressed?>` inside a forever loop (instead of separate `when key pressed` scripts) lets the player move diagonally and feels much smoother in a maze.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Match your walls: click the colour patch in `touching color` and pick the exact colour you painted the walls.

Expected: The player only resets on your real wall colour.

2. Tune the speed: if the player slips through thin walls, lower the moves to (2); if it feels sluggish, raise them to (4).

Expected: Smaller steps are more precise; larger steps are faster.

3. Add a start label: after `go to x: -210 y: -150` in the flag script, add `say (Reach the top-right!)`.

Expected: The player explains the goal at the start of each run.

Project 2: The goal and winning

Add a goal to reach. When the player touches it, it shouts a `broadcast [win]` message. The stage hears it and switches to a win screen, and the goal hears it and cheers. One message, two listeners - that is how sprites work as a team.

Get ready. Add a **Goal** sprite (click *Choose a Sprite*, pick a star or ball, rename it **Goal**) and drag it to the top-right of the maze. Add a second backdrop and paint *YOU WIN* on it; name that backdrop **win**.

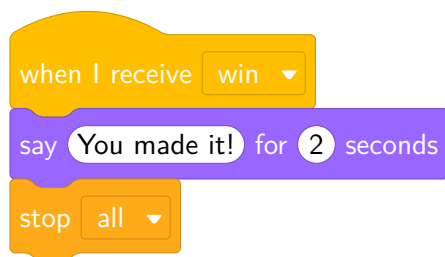
New blocks.

<code>broadcast [win]</code>	A gold Events block. It shouts a message to every sprite and the stage at once.
<code>when I receive [win]</code>	A gold hat. The script under it runs when that message is shouted.
<code>switch backdrop to [win]</code>	Change the stage picture - here, to the win screen.

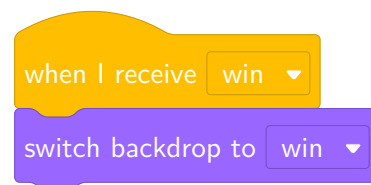
Build it.

1. On the **Player**, inside the forever loop, add `if <touching [Goal]>` then `broadcast [win]`. Click the arrow in the broadcast block and choose *New message*, name it **win**.
2. On the **Goal**, add a new script: `when I receive [win]`, then `say (You made it!) for (2) seconds` and `stop [all]`.
3. On the **Stage**, add `when I receive [win]` then `switch backdrop to [win]`.
4. Green flag, steer to the goal. The win screen appears, the goal cheers, and the game stops.

One message, two listeners - the Goal (left) and the Stage (right):



On the Goal sprite



On the Stage

`broadcast` does not say who should react - it just shouts. Any sprite with a matching `when I receive` hat answers. Add more listeners any time without touching the sprite that broadcasts.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Give winning a sound: on the Goal, add `start sound [Cheer]` before the say block in the `when I receive [win]` script.

Expected: A cheer plays the moment the player wins.

2. Make the goal inviting: add a `when green flag clicked` script on the Goal with a `forever` loop that does `change size by (5)`, `wait (0.1) seconds`, `change size by (-5)`, so it pulses.

Expected: The goal gently grows and shrinks to draw the eye.

3. Hide the win screen at the start: on the Stage, add `when green flag clicked` then `switch backdrop to [maze]` so each new game begins on the maze.

Expected: The maze always shows first; the win screen only on a win.

Project 3: The enemy and losing

Add danger. A patrolling enemy slides up and down the maze, and a `lives` counter starts at 3. Touch the enemy and you lose a life and restart; run out of lives and the game broadcasts `lose`.

Get ready. Add an **Enemy** sprite (rename it **Enemy**) and place it on the path. Make a variable **lives** *For all sprites*. Add a backdrop with *GAME OVER* painted on it, named **lose**.

New blocks.

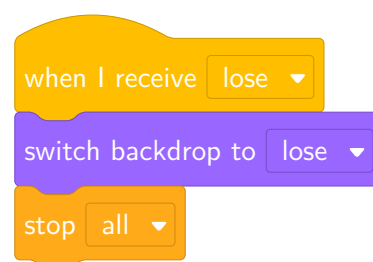
<code>glide () secs to x: () y: ()</code>	Slides the sprite smoothly to a spot over a set time - perfect for a patrol.
<code>broadcast [lose]</code>	A second message, shouted when the player runs out of lives.

Build it.

1. On the **Enemy**, make it patrol: `when green flag clicked`, then a `forever` loop with `glide (1) secs to x: (100) y: (120)` and `glide (1) secs to x: (100) y: (-120)`.
 2. On the **Player**, in the `when green flag clicked` script (before the loop), add `set [lives] to (3)`.
 3. In the Player's `forever` loop, add `if <touching [Enemy]?>` then `change [lives] by (-1)`, `go to x: -210 y: -150`, and `wait (0.5) seconds`.
 4. Just below that, still in the loop, add `if <(lives) = (0)>` then `broadcast [lose]`. Make the new **lose** message.
 5. On the **Stage**, add `when I receive [lose]` then `switch backdrop to [lose]` and `stop [all]`.
 6. Green flag. Dodge the enemy - three touches and it is game over.
- The enemy's patrol (left) and the Stage's lose handler (right):



Enemy: patrol up and down



Stage: game over

The lives check uses `(lives) = (0)` - the same green comparison from Scratch C. When it is true, one `broadcast [lose]` ends the game everywhere at once.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Show the danger: on the Player, in the touching-Enemy part, add `say (Ouch!) for (0.3) seconds` before the restart.

Expected: The player reacts each time the enemy catches it.

2. Make the enemy faster: change both glide times from (1) to (0.6) seconds.

Expected: The enemy patrols quicker, so it is harder to slip past.

3. Add a second enemy: right-click the Enemy and choose *duplicate*, then change the copy's glide x to a different column so it guards another part of the maze.

Expected: Two enemies patrol different lanes.

Project 4: Mini project: screens and the game loop

Wrap the game in a proper start. A title screen waits for the player; press space to begin and a `broadcast [start]` kicks everything off. Now your game has real states - title, playing, win, lose - all steered by messages.

Get ready. Add a backdrop with *MAZE QUEST - press space* painted on it, named **title**. You will move the player's start-up so it waits for the **start** message instead of the green flag.

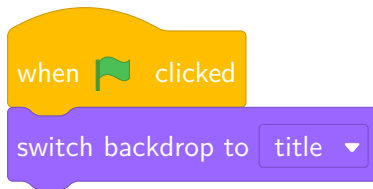
New blocks.

`when I receive [start]` Begin play only after the title screen, not the instant the flag is clicked.

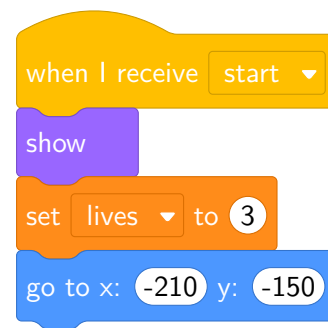
Build it.

1. On the **Stage**, add `when green flag clicked` then `switch backdrop to [title]`, so every game opens on the title.
2. On the **Player**, add `when green flag clicked` then `hide` (the player waits off-screen on the title).
3. Add a tiny starter: `when [space] key pressed` then `broadcast [start]` (make the **start** message).
4. Change the Player's set-up to run on the message: `when I receive [start]`, then `show`, `set [lives] to (3)`, `go to x: -210 y: -150`, and the `forever` movement loop you built in Projects 1-3.
5. On the **Stage**, in the `when I receive [start]` script, `switch backdrop to [maze]` so play begins on the maze.
6. Green flag: the title shows. Press space and the maze, player, enemy, and goal all spring to life. You have a complete game!

The title setup (left) and the player starting on the message (right):



Stage: show the title



Player: begin on start

Messages are the game's traffic controller: `start` begins play, `win` and `lose` end it. Each sprite just listens for the messages it cares about - that is how big projects stay organised.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Add Level 2: paint a harder maze backdrop named `level2`. Change the Goal's `when I receive [win]` to `broadcast [next]` instead of `stop [all]`, and add `when I receive [next]` handlers that switch the backdrop to `level2` and move the goal.

Expected: Winning loads the next level instead of ending the game.

2. Keep a high score: make a variable `best`. When the player wins, add `if <(lives) > (best)>` then `set [best] to (lives)` so the best finish is remembered.

Expected: The best result so far stays on screen across games.

3. Add title music: on the Stage, in `when green flag clicked`, add `start sound [Dance Around]` so the title screen has a tune.

Expected: Music plays while the player reads the title.